Affordable Mistakes: Severity-Aware Multiparty Session Types for Participants that Choose Wrongly*

Anonymous author
Anonymous affiliation

Anonymous author
Anonymous affiliation

Abstract

Multiparty session types guarantee that well-typed participants interact correctly, on the hypothesis that each participant *is* its type. A participant that was instructed rather than compiled — a language-model agent — breaks the hypothesis in one specific way: at a branch where it should select A it may select B. Forbidding this is impossible, and would be useless if it were possible. We change what the discipline is for. It does not prevent wrong choices; it prevents *catastrophic* ones and reports the rest as risk. The fault model is the participant's own selection at a branch, defined through the guards that asserted global types already carry. The consequence of a wrong selection is classified by *outcome* against a STRIPS-style world: goal still reachable, goal lost but nothing harmed, or a hazard reachable through a *point of no return*. Cascading is a budget k of misselections, with no probabilities anywhere. The contribution is a *well-formedness condition* on global types, k-misselection tolerance, with the standard MPST payoff: a session typed against a k-tolerant protocol by the base discipline's direct-conformance judgment is hazard-free on every run with at most k misselections, and budgets distribute over participants. The condition is exactly characterized by one syntax-directed rule, composes modularly across sequential composition, and is decidable for regular protocols by an executable, verified decision procedure. All of this is mechanized in Coq, axiom-free. An implementation over an existing achievability compiler shows that two existing corpora contain no decision points or irreversible tools at all — the question is invisible to them — and, on a purpose-built benchmark of fifteen protocols, classifies forty branches, finds seven 0-tolerant protocols each naming its point-of-no-return action, verifies three repairs, and demonstrates that modular checking with the interface abstraction the theory licenses is linear where whole-system analysis is exponential.

2012 ACM Subject Classification Theory of computation → Type structures; Theory of computation → Process calculi; Software and its engineering → Software verification

Keywords and phrases Session types, Multiparty, Behavioural types, Safety, Irreversibility, Fault tolerance, Mechanized metatheory

Digital Object Identifier 10.4230/LIPIcs.ECOOP..0

Supplementary Material Anonymous supplementary material

Funding Anonymous funding

Acknowledgements Anonymous acknowledgements

WIP: Status. Everything in Sections 5–9 is mechanized (`proof/`, 44 results, all `Print Assumptions` closed). The evaluation numbers in Section 10 are real (`scripts/severity_eval.py`, results in `paper/WIP/results/`). Scope caveats are stated where they apply. The predecessor draft was withdrawn after its own mechanized audit found it vacuous (Section 13).

* WORKING DRAFT — not for submission. Base rules follow `paper/tas.tex`, authoritative for Section 3.

1 Introduction

Multiparty session types (MPST) guarantee communication safety, session fidelity and deadlock-freedom for systems whose participants *are* their local types [1]: the code was checked, so every run is a compliant run. A participant that was instructed rather than compiled breaks that hypothesis in a specific way. At a choice point it may take branch B where the situation demanded branch A — not by violating the protocol (both branches are in the type) but by judging the situation wrongly. This is the ordinary condition of a language-model agent occupying a role, and it is why such agents are employed: their latitude.

The tempting response is to make the type system prevent it. That is doubly wrong: impossible, because the judgement is the participant's, not the protocol's; and undesirable, because a participant that may never choose wrongly is one that may never choose. What is needed is *triage*.

The reframing. Do not ask whether the participant complied. Ask *what a wrong choice costs*. Three answers matter, and the middle one is what existing disciplines cannot express (Figure 1):

- Benign — a detour; the goal is still reachable.
- Futile — the goal is lost, but nothing is harmed. *Failure is not disaster.*
- Catastrophic — a hazard becomes reachable.

A system that blocks Futile is useless; one that permits Catastrophic is dangerous. Separating them is the whole job. Thesis: *session types say what may happen; this says what you can survive.*

What is and is not new. The ingredients are old; the discipline is not. Branch guards on global types are design-by-contract MPST [2, 4, 5], and guard violation on selection is the alarm condition of the monitoring line [3]. Whether a goal remains reachable is dead-end detection in planning [29, 30]; whether an effect can be undone is undoability checking [31, 32]. Bounded fault tolerance is classical in planning [27, 28] and in MPST with crash-stop failures [8, 9]. A syntax-directed system equivalent to a model checker is a known template [21, 22]. We claim none of these. We claim one thing:

> *A well-formedness condition on global types — k-misselection tolerance — whose fault model is the participant's own selection at a branch, whose analytic payload is a classification of each wrong selection by outcome against a world, and which delivers the standard MPST guarantee: a session typed against a k-tolerant protocol is hazard-free within budget k, with budgets distributing over participants. The condition is exactly characterized, modular, decidable for regular protocols, and mechanized end to end.*

We are careful with the word *type system*. The condition is on global types, as well-assertedness [2] and deadlock-freedom conditions are; the participants are typed by the base discipline's direct-conformance judgment, unchanged. The type-theoretic content is the theorem that couples the two (Section 7), not the rule that decides the condition.

Contributions.

C1 *Misselection and severity* (§4, §5): guard failure as the fault model; a budgeted semantics; the Benign/Futile/Catastrophic partition; the point of no return.

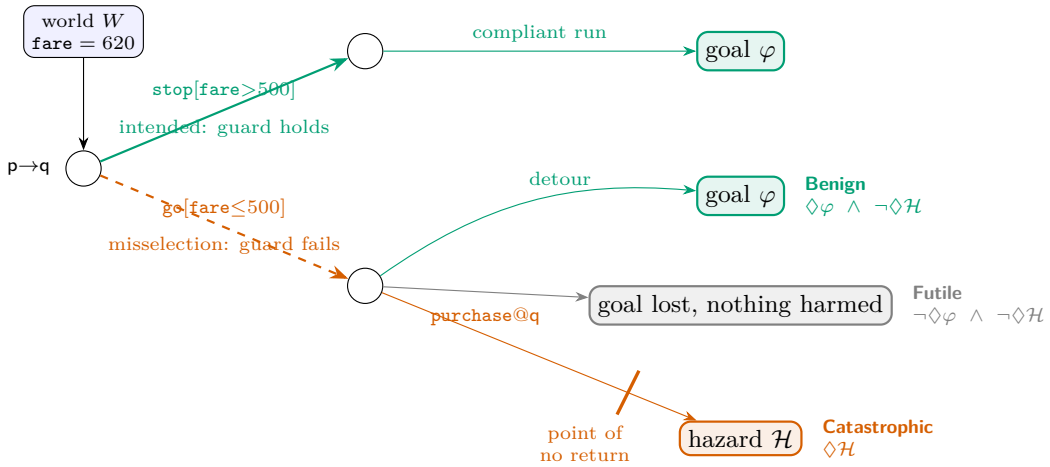

Figure 1 The idea. At a guarded choice in world W, the branch whose guard holds is *intended*; taking another is a *misselection*. Its severity is a property of the residual: Benign if the goal is still reachable, Futile if the goal is lost but no hazard is reachable, Catastrophic if a hazard is reachable — typically by crossing a *point of no return*, an irreversible capability after which the goal cannot be recovered.

C2 *Exact characterization* (§6): one rule, T-Choice-Safe, sound and complete for k-bounded hazard-freedom, so a failure is a genuine risk (✓ `TC_exact`).

C3 *From protocols to programs* (§7): a session typed against a k-tolerant protocol is hazard-free on every run of cost at most k (✓ `bridge_run`); per-role allowances summing to k are jointly tolerated (✓ `budget_distributes`).

C4 *Regular protocols* (§8): loops, the product construction, the bound on loop re-entry, and an executable decision procedure proved sound and complete (✓ `decide_reachb_correct`).

C5 *Modularity and evaluation* (§9, §10): sequential composition is typed against an interface (✓ `TC_seq_interface`); an implementation and a severity benchmark show the interface abstraction is what makes modular checking pay, and that existing corpora cannot even pose the question.

A design expectation was refuted by the mechanization and a benchmark finding refines it: tolerance is anti-monotone in the capability context for a fixed hazard (✓ `tolerance_antitone_in_ctx`), yet a new tool that re-establishes a lost atom can *remove* a point of no return (§10).

2 Overview

Running example. A planner `p` and a worker `q` holding a purchase tool. Atoms `verified`, `booked`, `funds`; `verify` adds the first, `purchase` adds the second and deletes the third, and no tool restores `funds`. The protocol offers two branches:

$$G_{\mathsf{bad}} \;=\; \mathsf{p}\to\mathsf{q} : \{\ \mathtt{safe}.\ \mathtt{verify@q}.\ \mathtt{purchase@q}.\ \mathsf{End}\ ,\ \ \mathtt{fast}.\ \mathtt{purchase@q}.\ \mathsf{End}\ \}$$

with goal $\varphi = \mathtt{booked} \wedge \mathtt{verified}$. Under the default instantiation (§4) `safe` is intended because the goal survives it and `fast` is a misselection because it does not. The tool reports, and the Coq development proves for the same instance: G_{bad} is 0-tolerant (✓ `Gbad_is_0_tolerant`) — if the participant never misselects, nothing bad happens, which is where classical MPST stops; it is *not* 1-tolerant (✓ `Gbad_not_1_tolerant`) — one wrong choice purchases unverified,

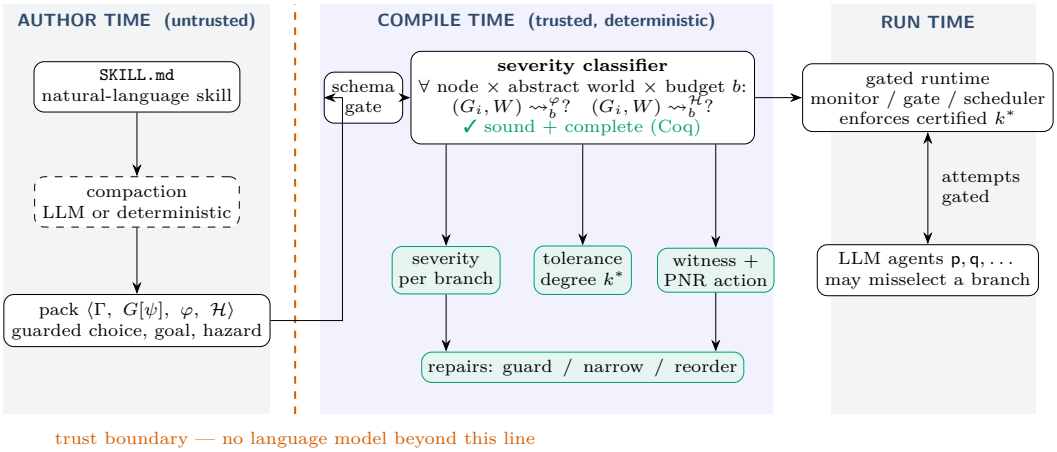

Figure 2 Architecture. Compaction from prose to a pack is untrusted and may use a language model; nothing beyond the trust boundary does. The severity classifier runs the discipline's existing reachability search pointwise at every branch of every choice, at every abstract world and budget, and emits a per-branch severity, the tolerance degree k^*, a witness naming the point-of-no-return action, and repair suggestions. The certified pack drives the gated runtime that refuses out-of-protocol attempts.

and `purchase` is the point of no return; and narrowing away `fast` is tolerant at every k (✓ `Ggood_is_k_tolerant`). The headline number is the *tolerance degree* k^*, the largest k the protocol survives.

For a protocol with two decision points the tool's output is what an engineer needs (Section 10, `migration_backup`):

```
migration_backup: tolerance degree k* = 1  (irreversible ['drop_old'])
  /choice@ops#0                              skip_backup  Catastrophic (misselection) PNR=drop_old
  /choice@ops#0/skip_backup/choice@ops#1  drop_old     Catastrophic (misselection) PNR=drop_old
  first catastrophe within budget 2: choose/skip_backup > act/migrate
                                   > choose/drop_old > act/drop_old
  repair (narrow): remove skip_backup at /choice@ops#0, ...
```

"Tolerant of one wrong choice; two wrong choices reach data loss, here is the path and the action that burns the bridge" is actionable. "Impossible" is not.

Architecture. Figure 2: a compile-time check on the pack, before any participant runs, with no language model in the trusted core — the base discipline's trust boundary. The runtime gate is inherited unchanged: it refuses *out-of-set* attempts (labels the protocol does not offer) before delivery, so those change no state. The deviation this paper analyses is *in-set*: a branch the type permits and the situation forbids.

3 The Base Discipline

We build on a goal-marked, capability-guarded synchronous multiparty discipline; `tas.tex` is authoritative and this section fixes notation. Predicates $p \in \mathsf{Pred}$, numeric variables $x \in \mathsf{Var}$, roles p, q, capabilities $a \in \mathsf{Cap}$, labels $\ell \in \mathsf{Lab}$. The state logic is quantifier-free linear integer arithmetic with uninterpreted booleans; a world $W = \langle B, N \rangle$ valuates it. A capability context maps each possessed tool to a guarded STRIPS effect $\Gamma(a) = \langle pre_a, \mathit{eff}_a \rangle$, $\mathit{eff}_a = \langle Add_a, Del_a, Asg_a, Nd_a \rangle$. Firing is World-Act: $\langle W \rangle\, a\, \langle W' \rangle$ iff $W \vDash pre_a$ and

$W' \in \mathrm{apply}(\mathit{eff}_a, W)$. Global types are coinductive regular trees

$$G \ ::=^{coind} \ \mathsf{p}{\to}\mathsf{q} : \{\ell_i.\, G_i\}_{i\in I} \ \mid \ a@\mathsf{p}.\, G \ \mid \ \checkmark\varphi.\, G \ \mid \ \mathsf{End}$$

and sessions $\mathbb{M} = \mathsf{p}_1[P_1] \parallel \cdots$ of processes $P ::=^{coind} \mathbf{0} \mid \mathsf{p}!\{\ell_i.P_i\} \mid \mathsf{p}?\{\ell_i.P_i\} \mid a.P$ are typed *directly* against G by the coinductive judgment $G \vdash (\mathbb{M}; W)$ — no local types, projection, or separate subtyping [16, 17] — whose T-Comm requires the sender to offer exactly the protocol's labels, the receiver at least them, and every branch to check against the same residual. Achievability is existential may-reachability of a goal-satisfying configuration. The base metatheory (subject reduction, session fidelity, refutation soundness) is mechanized for the finite fragment. This paper adds one annotation to choice, one counter to the semantics, and one well-formedness condition.

4 Misselection and the Budget

▶ **Definition 1** (Guarded choice). *A choice node carries, per branch, the predicate that makes that branch the intended one:* $\mathsf{p}{\to}\mathsf{q} : \{\ell_i[\psi_i].\, G_i\}_{i\in I}$. *In world* W*, branch* i *is* intended *if* $W \vDash \psi_i$*; taking a branch with* $W \nvDash \psi_i$ *is a* misselection.

The syntax is that of asserted global types [2], whose well-formedness conditions guarantee that a compliant participant can always pick a satisfied branch. We keep the syntax and drop the guarantee: the guards are not a constraint on an implementation but the *definition of wrong*.

Default instantiation. Guards need not be written. When a branch carries none, the implementation uses the *rational-choice* default: a branch is intended in W iff the goal is still reachable from its residual. Under this default a misselection is a choice that forfeits the goal, and once the goal is forfeit every subsequent choice is a misselection — there is no longer a right one — so the budget counts decisions taken after the point at which the task was lost. Explicit guards override the default where the intended branch is a matter of policy rather than reachability (Section 10 uses both).

▶ **Definition 2** (Budgeted reachability). $(G, W) \rightsquigarrow_b^{\mathcal{H}}$ *holds when a hazard state is reachable from* (G, W) *using at most* b *misselections (Coq:* `reach_haz`*): the head rules of the base semantics, plus*

$$\textit{RH-Comm-Ok}\ \frac{\ell_i[\psi_i].G_i \in \mathit{brs} \qquad W \vDash \psi_i \qquad (G_i, W) \rightsquigarrow_b^{\mathcal{H}}}{(\mathsf{p}{\to}\mathsf{q} : \mathit{brs}, W) \rightsquigarrow_b^{\mathcal{H}}}$$

$$\textit{RH-Comm-Dev}\ \frac{\ell_i[\psi_i].G_i \in \mathit{brs} \qquad W \nvDash \psi_i \qquad (G_i, W) \rightsquigarrow_b^{\mathcal{H}}}{(\mathsf{p}{\to}\mathsf{q} : \mathit{brs}, W) \rightsquigarrow_{b+1}^{\mathcal{H}}}$$

Compliant progress is free; a misselection costs one unit. Choices marked as controlled by the environment are resolved demonically at no cost.

▶ **Definition 3** (k-misselection tolerance). G *is* k-misselection-tolerant *for hazard* $\mathcal{H}$ *under* Γ *from* W_0 *if* $\neg\,(G, W_0) \rightsquigarrow_k^{\mathcal{H}}$.[1] 0*-tolerance is classical MPST safety.*

[1] We avoid two established names. In MPST, *k-safety* is a buffer bound [14]; in hyperproperties it is a property of k traces [15]. *k-resilient* plans survive k action failures [27]; our fault is the participant's own selection and our target is a hazard, not the goal.

The budget is possibilistic: a design parameter of the deployment, not a measurement of the participant. Every theorem below holds for every participant that misselects at most k times, however it does so.

5 Severity and the Point of No Return

Let φ be the goal and $\mathcal{H}$ the hazard; $\rightsquigarrow_b^{\varphi}$ is budgeted goal reachability.

▶ **Definition 4** (Severity). *For a residual (G, W) at budget b: Catastrophic if $(G, W) \rightsquigarrow_b^{\mathcal{H}}$; otherwise Futile if $\neg(G, W) \rightsquigarrow_b^{\varphi}$; otherwise Benign.*

▶ **Theorem 5** (Partition). *The classes are pairwise disjoint, and exhaustive whenever the two reachability questions are decidable.* ✓ `severity_disjoint`, ✓ `severity_exhaustive`

The content is the separation of Futile from Catastrophic. In model-based safety assessment [35] severity is supplied by the analyst; here it is computed from goal reachability.

▶ **Definition 6** (Point of no return). *(G, W) is a point of no return for φ if $\neg(G, W) \rightsquigarrow_b^{\varphi}$. An action is* goal-destroying *at (G, W) if the goal is reachable before it fires and at no successor.*

Default hazard. Because effects are STRIPS, an atom in Del_a is restored only by some b with the atom in Add_b; if Γ has no such b, a is *irreversible*. The implementation's default hazard, needing no annotation, is *burning a bridge while lost*: an irreversible action fires from a configuration after which the goal is unreachable. Tools whose real-world effect cannot be undone but whose pack models no delete-list (an email cannot be unsent) are declared irreversible explicitly. This is dead-end detection [29] and undoability checking [31] lifted to a protocol; unrestricted reversibility checking is harder than planning [32], and our decidability rests on the widened finite-range fragment the checker already commits to.

6 The Well-Formedness Condition, Exactly

$\vdash_b G, W$ reads: from (G, W), no hazard arises under any run with at most b misselections.

$$\text{T-End}\ \frac{W \not\models \mathcal{H}}{\vdash_b \mathsf{End}, W} \qquad \text{T-Goal}\ \frac{W \not\models \mathcal{H} \qquad \vdash_b G, W}{\vdash_b \checkmark\varphi.G, W}$$

$$\text{T-Act}\ \frac{W \not\models \mathcal{H} \qquad \forall W'.\ \langle W\rangle\, a\, \langle W'\rangle \implies \vdash_b G, W'}{\vdash_b a@\mathsf{p}.G,\ W}$$

$$\text{T-Choice-Safe}\ \frac{\begin{array}{c} W \not\models \mathcal{H} \\ \forall i \in I.\ \ W \models \psi_i \implies \vdash_b G_i, W \\ \forall i \in I.\ \ W \not\models \psi_i \implies \forall c.\ b = c{+}1 \implies \vdash_c G_i, W \end{array}}{\vdash_b\ \mathsf{p}{\to}\mathsf{q} : \{\ell_i[\psi_i].G_i\}_{i\in I},\ W}$$

An intended branch is checked at the same budget; a misselectable branch at one less, and not at all when the budget is exhausted. This is "affordable mistake" as a rule.

▶ **Theorem 7** (Exact characterization). *$\vdash_k G, W \iff \neg(G, W) \rightsquigarrow_k^{\mathcal{H}}$, hence a branch is Catastrophic iff its residual fails the condition.* ✓ `TC_sound`, ✓ `TC_complete`, ✓ `TC_exact`

We do not present this as deep — the rules are the reachability relation with the negation pushed through, and the proof is short. Its value is that a failure is never an artifact of conservative rules, which preserves the refutation asymmetry the discipline is built on, and that the condition is syntax-directed, which is what §9 exploits. Two monotonicity facts complete the picture: tolerance is downward closed in k (✓ `tolerance_downward_closed`), and reachability is monotone in Γ, so for a *fixed* hazard granting a tool can only lower k^* (✓ `tolerance_antitone_in_ctx`) — the formal case for least privilege, and a refutation of our own prior expectation that more tools mean more recovery.

Not resource-aware typing. The budget resembles the potential of resource-aware session types [23, 24] and graded modalities [25]; the resemblance misleads. Potential is consumed by the program's own execution and exhaustion is a type error. The budget is consumed by an *adversarial* choice the program does not make, so the rule quantifies over deviations rather than amortising cost, and exhaustion is unconstrained.

7 From Protocols to Programs

Everything so far concerns the global type's semantics. The theorem that makes it a discipline about *programs* couples the condition to the base judgment $G \vdash (\mathbb{M}; W)$, lifted to guarded choice without touching the guards: T-Comm still requires the sender to offer exactly the protocol's labels and every branch to check against the same residual; which label the participant eventually picks is its own business.

▶ **Definition 8** (Instrumented head-move semantics). *A session step carries the acting role and its cost:* 0 *for an action or an intended selection,* 1 *when the sender picks a label whose guard fails in the current world (Coq:* `hstep`*). A run records the trace of (role, cost) events;* total *sums costs and* percost_r *sums those of role* r*.*

▶ **Theorem 9** (Bridge). *If* $G \vdash (\mathbb{M}; W)$ *and* $\vdash_k G, W$*, then every instrumented head step of cost* $c \le$ *the remaining budget preserves typing and debits the budget by exactly* c *(✓* `bridge_step`*); consequently, along every run of total cost at most* k*, no configuration is a hazard state (✓* `bridge_run`*).*

▶ **Theorem 10** (Budgets distribute over participants). *Give each role* r *an allowance* k_r*. If the allowances of the roles that act sum to at most* k*, a session typed against a* k*-tolerant protocol is hazard-free on every run in which each role stays within its own allowance.* ✓ `budget_distributes`

Theorem 10 is the cross-participant composition result: the global budget is a contract that can be split among participants and checked per participant. Scope: the head-move semantics of the finite fragment, the same scope as the base subject-reduction mechanization; extending both to bystander interleavings is joint future work.

8 Regular Protocols

A regular global type unfolds to a finite graph of protocol nodes; over the finite abstract world of the widened fragment, budgeted reachability is reachability in the product $\mathit{Node} \times \mathit{World} \times \{0..k\}$. The mechanization is abstract over any finite node and world types with computable successors (Coq: `Regular.v`).

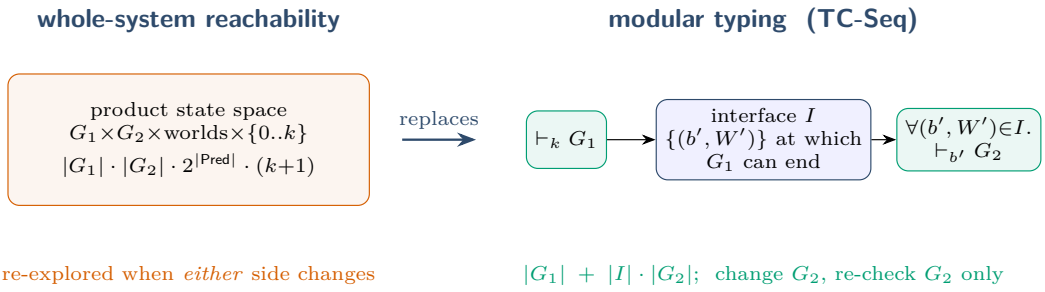

Figure 3 Modularity. A whole-system check of $G_1; G_2$ explores the product of both protocols with the world and the budget and re-explores it when either changes. TC-Seq types G_2 only at the interface G_1 exposes; the interface form lets that interface be a declared invariant coarser than the concrete exit set.

▶ **Theorem 11** (Product). $(n, w) \rightsquigarrow_b^{\mathcal{H}}$ *iff a hazard state is reachable from* (n, w, b) *in the product graph, whose misselection edges decrement the third component (✓* `product_correspondence`*).*

▶ **Theorem 12** (Loops meet the budget). *The budget component never increases along a product path (✓* `budget_never_increases`*); a path with* d *misselection edges from budget* b *has* $d \leq b$*, and exactly* $b - b'$ *if it ends at budget* b' *(✓* `dev_edges_bounded`*, ✓* `dev_edges_exact`*). A loop containing a misselectable branch is therefore traversed wrongly at most* k *times within budget* k*.*

▶ **Theorem 13** (Decidability). *For finite node and world types with computable successors, the boolean* `decide_reachb` *decides* $(n, w) \rightsquigarrow_k^{\mathcal{H}}$*: it is sound (✓* `decide_sound`*) and complete (✓* `decide_complete`*), the latter via a pigeonhole argument that reachability is witnessed by a path no longer than the reachable-state count (✓* `reach_bounded_path`*); together, ✓* `decide_reachb_correct`*.*

▶ **Proposition 14** (Tolerance degree). k^* *is computable. Whether* $k^* = \infty$ *is plain reachability in Node* $\times$ *World with misselection edges made free; otherwise* k^* *is at most the number of misselection edges on a shortest hazard path, bounded by the product size, and is found by iterating Theorem 13. With* $|World| = 2^{|\text{Pred}|}$ *the decision problem is in PSPACE; each iteration is linear in the product.*

Proof. Paper proof from Theorems 11–13: dropping the budget component yields the ∞ case; `dev_edges_exact` bounds k^*; reachability in a succinctly presented graph is in PSPACE. ◀

9 Why a Discipline, and Not a Model Checker

Theorem 7 invites the objection that the rules are a decision procedure in disguise. The objection is old and Naik and Palsberg [21] gave the standard reply. We rest instead on a theorem and an experiment.

▶ **Theorem 15** (Modular composition). *If* $\vdash_k G_1, W$ *and* $\vdash_{b'} G_2, W'$ *for every* (b', W') *at which* G_1 *can end from budget* k*, then* $\vdash_k G_1; G_2, W$ *(✓* `TC_seq`*). In interface form: for any* I *containing those exit points, it suffices that* $\vdash_{b'} G_2, W'$ *for all* $(b', W') \in I$ *(✓* `TC_seq_interface`*).*

The theorem is what a whole-system reachability run cannot be: k becomes a contract on a boundary. But it does not, by itself, say the boundary is small, and the evaluation shows it is not: the *concrete* exit set grows with the number of distinguishable worlds, exponentially when every segment leaves its own atoms behind. What makes modular checking pay is the *interface* form with a coarser I — and the principled coarsening is the cone of influence, projecting exit worlds onto the atoms the remaining segments read, sound because dropped atoms cannot affect them. With that abstraction the modular check is linear where whole-system analysis is exponential (Section 10, Table 2). This is the axis on which MPST is currently being argued [18]; the community has accepted parameterising typing by a model-checked property [8, 9], and the theorem plus the projection is our concrete instance of that move.

10 Implementation and Evaluation

`skillc severity` implements the analysis over the existing achievability compiler: the checker's own symbolic search (predicates concrete, numerics symbolic, widened at loop heads) is run pointwise at every branch of every choice, at every abstract world and budget up to $k_{\max} = 4$, with the default guards and hazard of §4–§5 and explicit overrides where declared. It reports per-branch severity, k^*, the first hazard witness, the point-of-no-return action, and the narrowing repair. Seven tests check the tool's verdicts on the paper's instances against the Coq development's.

Finding 1: existing corpora cannot pose the question. We ran the analysis on the compiler's achievability corpus (15 packs), its extension (6), and the 17 public skills of the reference collection compiled by the deterministic front-end. Of these 38 packs, *none* declares an irreversible effect, and the 17 real skills contain *no choice point at all*: the front-end produces linear, add-only protocols. Every pack is trivially tolerant at every k. This is a result about modelling, not about the tool: tools' real-world irreversibility (sending, paying, deleting, deploying) is not captured by achievability-oriented compaction, so a severity analysis needs the declaration §5 introduces. It also means the question this paper asks has been invisible to the corpora the field evaluates on.

Finding 2: the severity benchmark. We therefore built fifteen protocols with genuine decision points and irreversible tools, each with a provenance note stating the expected verdict *before* running the tool: booking (the running example and its two repairs), database migration, an email campaign and its guard repair, deployment with and without rollback, file cleanup, order fulfilment with an environment-controlled bank, a retry loop with a destructive give-up, a guarded shipping choice, an insurance claim in two worlds, and a staged commit. Results (Table 1): 40 branches classified in 0.22 s total; 26 Benign, 3 Futile, 11 Catastrophic; k^* distribution 7×0, 2×1, $6 \times \geq 5$; point-of-no-return actions `purchase, send, deploy, delete, ship, purge, refund, drop_old, commit` — every one an externally visible effect. Fourteen of fifteen verdicts matched the pre-stated expectation; the fifteenth exposed an authoring error in our benchmark (a goal disjunct that failed to exclude `shipped`), corrected and recorded.

Three pairs carry the paper's claims. *Repairs restore tolerance:* reordering, narrowing and guarding each take a 0-tolerant protocol to tolerance at every k. *Severity is a property of node and world:* the insurance-claim protocol is identical in both rows; with an eligible claimant the wrong branch is Futile, with an ineligible one `refund` is a point of no return and

protocol	choices	loops	irreversible	k^*	Benign	Futile	Catastrophic
booking_fastpath	1	0	purchase	0	1	0	1
booking_reordered (repair: reorder)	1	0	purchase	≥ 5	2	0	0
booking_narrowed (repair: narrow)	1	0	purchase	≥ 5	1	0	0
migration_backup	2	0	drop_old	1	3	0	2
email_campaign	1	0	send (declared)	0	1	0	1
email_campaign_guarded (repair: guard)	1	0	send	≥ 5	2	0	0
deploy_with_rollback	1	0	—	≥ 5	1	1	0
deploy_no_rollback	1	0	deploy	0	1	0	1
file_cleanup	1	0	delete	0	1	0	1
order_fulfilment (environment choice)	3	0	ship	0	1	0	1
retry_then_purge	2	1	purge	0	6	0	1
shipping_detour (explicit guards)	1	0	—	≥ 5	2	0	0
claim_eligible	1	0	refund	≥ 5	1	1	0
claim_ineligible (same protocol)	1	0	refund	0	0	0	1
staged_commit	3	0	commit	1	3	1	2

Table 1 The severity benchmark ($k_{\max} = 4$; ≥ 5 means no hazard within four misselections). Total analysis time 0.22 s.

n	whole (complete) ms	exits	modular, concrete ms	interface	modular, projected ms	interface
1	18.6	3	17.7	3	18.6	2
2	108.6	6	66.1	8	38.4	2
3	494.5	12	197.5	20	67.1	2
4	2130.5	24	512.1	48	81.5	2
5	9217.6	48	1346.6	112	114.7	2
6	35752.0	96	2914.6	256	132.9	2

Table 2 Chains of the migration segment, $k = 2$. Re-check after replacing the last segment: whole-system 16–172 ms growing with n; modular 19–25 ms, constant. The deployment segment gives the same shape at smaller scale (728 ms vs 59 ms at $n = 6$).

$k^* = 0$. *The point of no return depends on* Γ*'s establishers:* with `rollback` available, `deploy` deletes nothing permanently and the wrong branch is merely Futile; remove `rollback` and the same branch is Catastrophic. The last pair refines Theorem 7's anti-monotonicity: for a *fixed* hazard more tools never help, but the derived hazard is not fixed — a tool that re-establishes a lost atom removes a point of no return.

Finding 3: modularity pays only with the interface abstraction. We chained a benchmark segment n times, $G; G; \cdots; G$ with atoms renamed per segment, and compared four analyses at $k = 2$ (Table 2): the whole-system *complete* enumeration of hazard-free terminations (like-for-like with computing an interface), modular checking with the *concrete* exit set, modular checking with exits *projected* onto the cone of influence of the remaining segments, and the cost of re-checking after the last segment is replaced by its unsafe variant. On the two-decision migration segment, whole-system analysis grows about fourfold per segment; the concrete interface also grows exponentially (256 points at $n = 6$) because each segment's choices leave distinct atoms behind; the projected interface has two points at every n and the analysis is linear. At $n = 6$ that is 35.8 s against 0.13 s. Re-checking after a change costs whole-system analysis the full run (172 ms) and modular analysis the last segment alone (22 ms, constant in n).

This is the honest form of the modularity claim. Theorem 15 does not make checking cheap; it makes it *decomposable*, and the interface form is what lets a coarse, declared invariant stand in for the concrete exits. Inferring good interfaces beyond the cone of influence is the

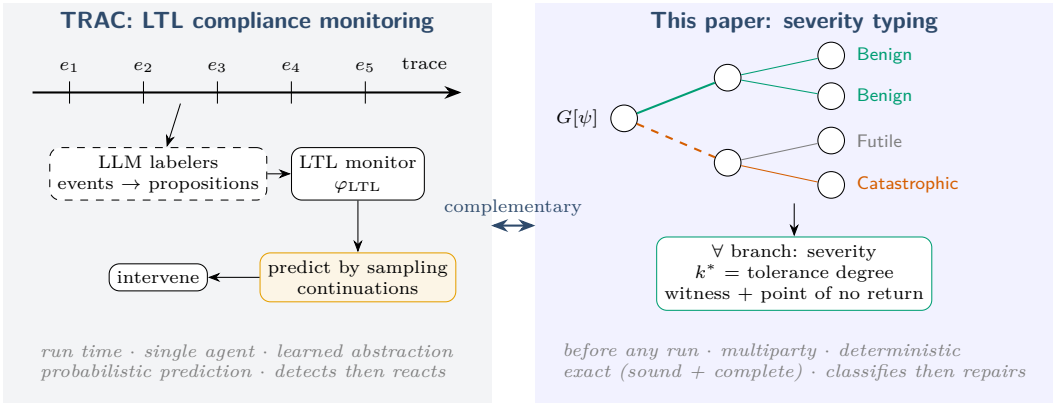

Figure 4 Two answers to "will the agent misbehave?". Left: TRAC observes a single agent's trace at run time, abstracts events to propositions with language-model labelers, checks an LTL constraint, predicts violations by sampling continuations, and intervenes. Right: this paper classifies every branch of a multiparty protocol before any run, deterministically and exactly. Complementary: TRAC's constraints can supply $\mathcal{H}$; its monitor can enforce at run time what the condition certified.

open engineering problem.

Not yet done. Predictive validity — running agents against the gated runtime and checking that hazard-reaching runs are exactly those exceeding k^* — is the experiment that would test the bridge theorem's applicability to a deployment; it requires live agents and is future work.

11 Repairs

When a branch is Catastrophic at the intended budget, four transformations make the mistake affordable: *guard* (insert a validation before the irreversible branch), *compensate* (add a recovery path), *narrow* (remove the branch, so the gate refuses it), and *reorder* (move the irreversible action after the validation). All have ancestors: amending asserted choreographies [36], interactional exceptions [37], supervisory control and shields [38, 39, 40], choreography repair [41, 42]. We record the one theorem we need and the three verified instances of Section 10.

▶ **Theorem 16** (Narrowing is sound). *Removing branches from a choice node preserves the condition at the same budget.* ✓ `repair_narrow_sound`

12 Relation to Runtime Compliance Monitoring

The closest work by goal is runtime compliance monitoring of language-model agents, of which TRAC [43] is the most developed: constraints in linear temporal logic, offline auditing, online monitoring, predictive monitors that sample continuations, intervening monitors that preempt a predicted violation. We share the question and differ on every mechanism (Figure 4, Table 3).

Two differences are deep. *Prediction versus projection:* TRAC must estimate whether a violation is coming by sampling a black box; here the guards fix the intended branch and reachability fixes the consequence of the others, so prediction is exact and free of any model of the agent, at the price of expressiveness. *The trusted seam:* TRAC's soundness

	TRAC [43]	this paper
when	run time	compile time, before any run
scope	one agent's trace	multiparty protocol
specification	LTL over propositions	protocol + world + goal + hazard
abstraction	language-model labelers	structured boundary, no LLM
prediction	sampling continuations	exact reachability
output	violation, witness, intervention	severity per branch, k^*, repair
guarantee	empirical violation-rate reduction	Theorems 7–15

Table 3 Axes on which the two approaches differ.

bottoms out in learned labelers whose temporal reasoning its own results show degrading with distance and constraint count; ours in a deterministic checker. The approaches compose. Choreographies with first-class agent choice mapped to games [45] own the quantitative version of this framing; we are qualitative and static.

13 What Changed, and Why

An earlier version typed *compliance* rather than *severity*: per-role trust modes, taint propagation, staleness grades. We mechanized its metatheory and it did not survive: a closure premise recursing at the same node terminated at bottom where only a quarantine rule with a partial residual applied, so nothing containing a capability was typable (✓ `act_vacuous_with_partial_qres`, coinductively too); its taint update ignored what a capability read (✓ `taint_laundering_refutes_noninterference`); its loop condition accepted cycles that never reset a grade (✓ `goal_only_cycle_hits_cap`). The audit is retained in the artifact. The present design has no closure premise, no taint, and no grades; deviation lives in the semantics as a budget. We report this because the negative result produced the positive one.

14 Related Work

MPST with a fault model. Crash-stop failures [8, 9, 10], affine and exceptional sessions [11, 12], and protocol-induced recovery [13] model participants that stop; misselection is a participant that continues, wrongly, and none classify the consequence or track a goal. Asserted and refined global types [2, 6, 4, 5] supply our syntax; monitoring [3] and blame [7] treat guard violation as an alarm rather than an object of analysis.

Bounded faults and dead ends. Fault-tolerant planning [28, 27] bounds action failures against goal reachability; we bound selections against hazard reachability. Dead-end detection [29, 30], undoability [31, 32] and excuse generation [33] supply the point-of-no-return machinery; reversibility in reinforcement-learning safety [34] shares the intuition without types.

Types and model checkers. Naik and Palsberg [21] and Kobayashi and Ong [22] set the template we instantiate. Resource-aware [23, 24] and graded [25] session types index judgments by a quantity, with the difference drawn in §6. Bounded-model-checking of fault-tolerant algorithms [26] is the verification-side analogue of Theorem 13.

Agents. Runtime enforcement and provenance gating of tool calls [46, 47, 48, 49], effect-typed agent languages [50], network-enforced session types [20], and LTL trace monitoring [43, 44] act on runs or in the host language; we classify branches before any run exists.

15 Limitations and Conclusion

Scope. The bridge theorem is mechanized for the head-move semantics of the finite fragment, the base mechanization's scope; regular protocols are mechanized abstractly over finite graphs, and connecting the two mechanizations through the unfolding of μ-types is the outstanding proof. A reviewer comparing scope against the mechanised subject-reduction development of [19] will be right to. The hazard is a declared or derived input; the discipline says what follows from it. Three of four repairs are argued, not mechanized. Benign inherits the checker's over-approximation; Catastrophic is a proof. The benchmark is ours, with expected verdicts stated in advance but authored by us. Predictive validity against live agents is not yet measured. Everything presumes the gated architecture.

Conclusion. A participant that may never err may never act. The useful question is not whether it will choose wrongly — it will — but whether the protocol around it survives the choice, and if not, what to change. Severity makes the question precise; the point of no return makes it computable with the machinery the checker already has; the budget makes cascading expressible without a probability; the bridge theorem carries the guarantee from the protocol to the typed session; and the interface makes the answer a contract rather than a run. The well-formedness condition reads: *every affordable mistake is allowed, and every unaffordable one is structurally unreachable.*

References

1 K. Honda, N. Yoshida, M. Carbone. Multiparty asynchronous session types. *POPL* 2008; *JACM* 63(1), 2016.
2 L. Bocchi, K. Honda, E. Tuosto, N. Yoshida. A theory of design-by-contract for distributed multiparty interactions. *CONCUR*, 2010.
3 L. Bocchi, T.-C. Chen, R. Demangeon, K. Honda, N. Yoshida. Monitoring networks through multiparty session types. *FMOODS/FORTE* 2013; *TCS* 669, 2017.
4 L. Gheri, I. Lanese, N. Sayers, E. Tuosto, N. Yoshida. Design-by-contract for flexible multiparty session protocols. *ECOOP*, 2022.
5 M. Vassor, N. Yoshida. Refinements for multiparty message-passing protocols: specification-agnostic theory and implementation. *ECOOP*, 2024.
6 F. Zhou, F. Ferreira, R. Hu, R. Neykova, N. Yoshida. Statically verified refinements for multiparty protocols. *OOPSLA*, 2020.
7 L. Jia, H. Gommerstadt, F. Pfenning. Monitors and blame assignment for higher-order session types. *POPL*, 2016.
8 A. D. Barwell, P. Hou, N. Yoshida, F. Zhou. Generalised multiparty session types with crash-stop failures. *CONCUR*, 2022; *LMCS*, 2025.
9 A. D. Barwell, P. Hou, N. Yoshida, F. Zhou. Designing asynchronous multiparty protocols with crash-stop failures. *ECOOP*, 2023 (Distinguished Paper).
10 K. Peters, U. Nestmann, C. Wagner. Fault-tolerant multiparty session types. *FORTE*, 2022.
11 S. Fowler, S. Lindley, J. G. Morris, S. Decova. Exceptional asynchronous session types. *POPL*, 2019.
12 N. Lagaillardie, R. Neykova, N. Yoshida. Stay safe under panic: affine Rust programming with multiparty session types. *ECOOP*, 2022.
13 R. Neykova, N. Yoshida. Let it recover: multiparty protocol-induced recovery. *CC*, 2017.

14 J. Lange, N. Yoshida. Verifying asynchronous interactions via communicating session automata. *CAV*, 2019.
15 M. R. Clarkson, F. B. Schneider. Hyperproperties. *J. Computer Security* 18(6), 2010.
16 S.-S. Jongmans, F. Ferreira. Synthetic behavioural typing: sound, regular multiparty sessions via implicit local types. *ECOOP*, 2023.
17 D. Castro-Perez, F. Ferreira, S.-S. Jongmans. *POPL*, 2026. [title to verify]
18 A synthetic reconstruction of multiparty session types. *PACMPL*, 2026. [authors to verify]
19 D. Tirore, J. Bengtson, M. Carbone. Multiparty asynchronous session types: a mechanised proof of subject reduction. *ECOOP*, 2025.
20 Larsen, Scalas, Amir, Jacobs, Wagemaker, Foster. NEST: network enforced session types. *ECOOP*, 2026.
21 M. Naik, J. Palsberg. A type system equivalent to a model checker. *ESOP* 2005; *TOPLAS* 30(5), 2008.
22 N. Kobayashi, C.-H. L. Ong. A type system equivalent to the modal mu-calculus model checking of higher-order recursion schemes. *LICS*, 2009.
23 A. Das, J. Hoffmann, F. Pfenning. Work analysis with resource-aware session types. *LICS*, 2018.
24 A. Das, D. Wang, J. Hoffmann. Probabilistic resource-aware session types. *POPL*, 2023.
25 D. Orchard, V.-B. Liepelt, H. Eades III. Quantitative program reasoning with graded modal types. *ICFP*, 2019.
26 I. Stoilkovska, I. Konnov, J. Widder, F. Zuleger. Verifying safety of synchronous fault-tolerant algorithms by bounded model checking. *TACAS*, 2019.
27 D. Aineto, A. Gaudenzi, A. Gerevini, A. Rovetta, E. Scala, I. Serina. Action-failure resilient planning. *ECAI*, 2023.
28 C. Domshlak. Fault tolerant planning: complexity and compilation. *ICAPS*, 2013.
29 M. Steinmetz, J. Hoffmann. Towards clause-learning state space search: learning to recognize dead-ends. *AAAI*, 2016.
30 J. Hoffmann, P. Kissmann, Á. Torralba. "Distance"? Who cares? Tailoring merge-and-shrink heuristics to detect unsolvability. *ECAI*, 2014.
31 J. Daum, Á. Torralba, J. Hoffmann, P. Haslum, I. Weber. Practical undoability checking via contingent planning. *ICAPS*, 2016.
32 Non-deterministic action reversibility: complexity results. *KR*, 2025. [authors to verify]
33 M. Göbelbecker, T. Keller, P. Eyerich, M. Brenner, B. Nebel. Coming up with good excuses: what to do when no plan can be found. *ICAPS*, 2010.
34 A. M. Turner, D. Hadfield-Menell, P. Tadepalli. Conservative agency via attainable utility preservation. *AIES*, 2020.
35 B. Bittner, M. Bozzano, R. Cavada, A. Cimatti, et al. The xSAP safety analysis platform. *TACAS*, 2016.
36 L. Bocchi, J. Lange, E. Tuosto. Three algorithms and a methodology for amending contracts for choreographies. *Sci. Ann. Comp. Sci.* 22(1), 2012.
37 M. Carbone, K. Honda, N. Yoshida. Structured interactional exceptions in session types. *CONCUR*, 2008.
38 P. J. Ramadge, W. M. Wonham. Supervisory control of a class of discrete event processes. *SIAM J. Control Optim.* 25(1), 1987.
39 R. Bloem, B. Könighofer, R. Könighofer, C. Wang. Shield synthesis: runtime enforcement for reactive systems. *TACAS*, 2015.
40 M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, U. Topcu. Safe reinforcement learning via shielding. *AAAI*, 2018.
41 S. Basu, T. Bultan. Automated choreography repair. *FASE*, 2016.
42 I. Lanese, F. Montesi, G. Zavattaro. Amending choreographies. *WWV*, 2013.
43 P. A. Alamdari, T. Q. Klassen, S. A. McIlraith. Formal methods meet LLMs: auditing, monitoring, and intervention for compliance of advanced AI systems. *FAccT*, 2026.
44 AgentLTL: a trace-verification framework for procedural compliance in tool-using LLM agents. *arXiv:2607.02599*. [authors to verify]

45 K. Gopinathan, J. Feser, et al. Pact: a choreographic language for agentic ecosystems. *arXiv:2605.03143*, 2026. [authors to verify]

46 H. Wang, C. M. Poskitt, et al. AgentSpec: customizable runtime enforcement for safe and reliable LLM agents. *ICSE*, 2026.

47 E. Debenedetti et al. Defeating prompt injections by design (CaMeL). *arXiv:2503.18813*, 2025.

48 M. Costa et al. Securing AI agents with information-flow control (FIDES). *arXiv:2505.23643*, 2025.

49 T. Shi et al. Progent: securing AI agents with privilege control. *arXiv:2504.11703*, 2025.

50 ETAS: an effect-typed language for agent systems. *arXiv:2607.17780*, 2026. [authors to verify]